Module 7: Support Vector Machine

Xiao Wang

Department of Statistics
Purdue University

Table of contents

1. Linear SVM

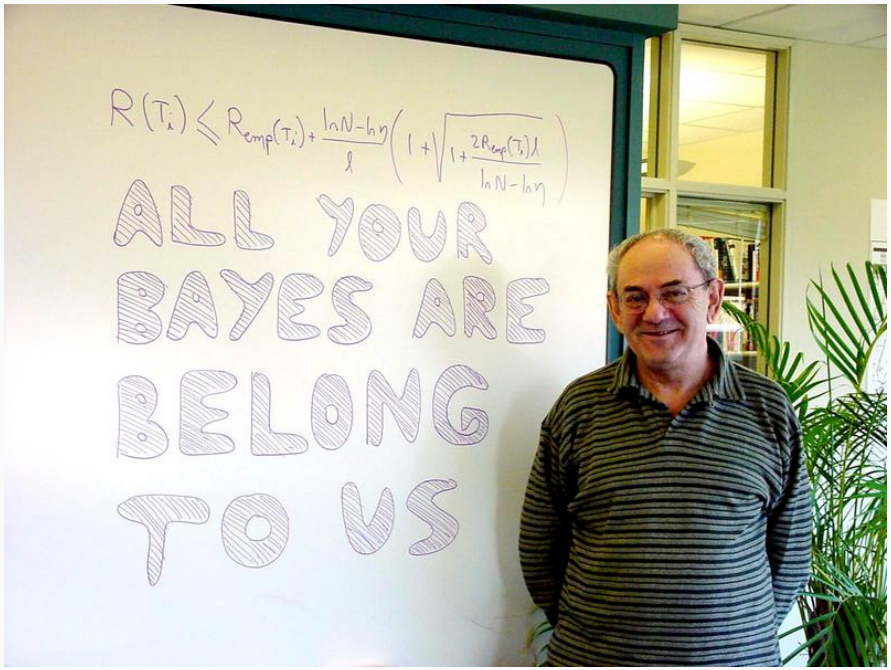
2. Nonlinear SVM

3. Multiclass SVM

Who invented SVM?

Vladimir Vapnik, born in 1936, studied Mathematics at Uzbek State University (then in Soviet Union) and moved to the US in 1990 to work with AT&T Bell Labs (now Nokia Bell Labs) shortly before publishing his major work in SVM.

His book Statistical Learning Theory (1998) has been outstandingly cited more than 60.000 times



Vapnik showing off his ERM framework with humor. Claiming superiority to Bayesian Statistics (with ERM formula at the top)

Vapnik also founded VC theory with Chervonenkis (Vapnik–Chervonenkis theory), a theory in computational learning.

Linear SVM

Support vector machines

Here we approach the two-class classification problem in a direct way:

We try to find a plane that separates the classes in feature space.

If we cannot, we get creative in two ways:

- We soften what we mean by "separates", and
- We enrich and enlarge the feature space so that separation is possible.

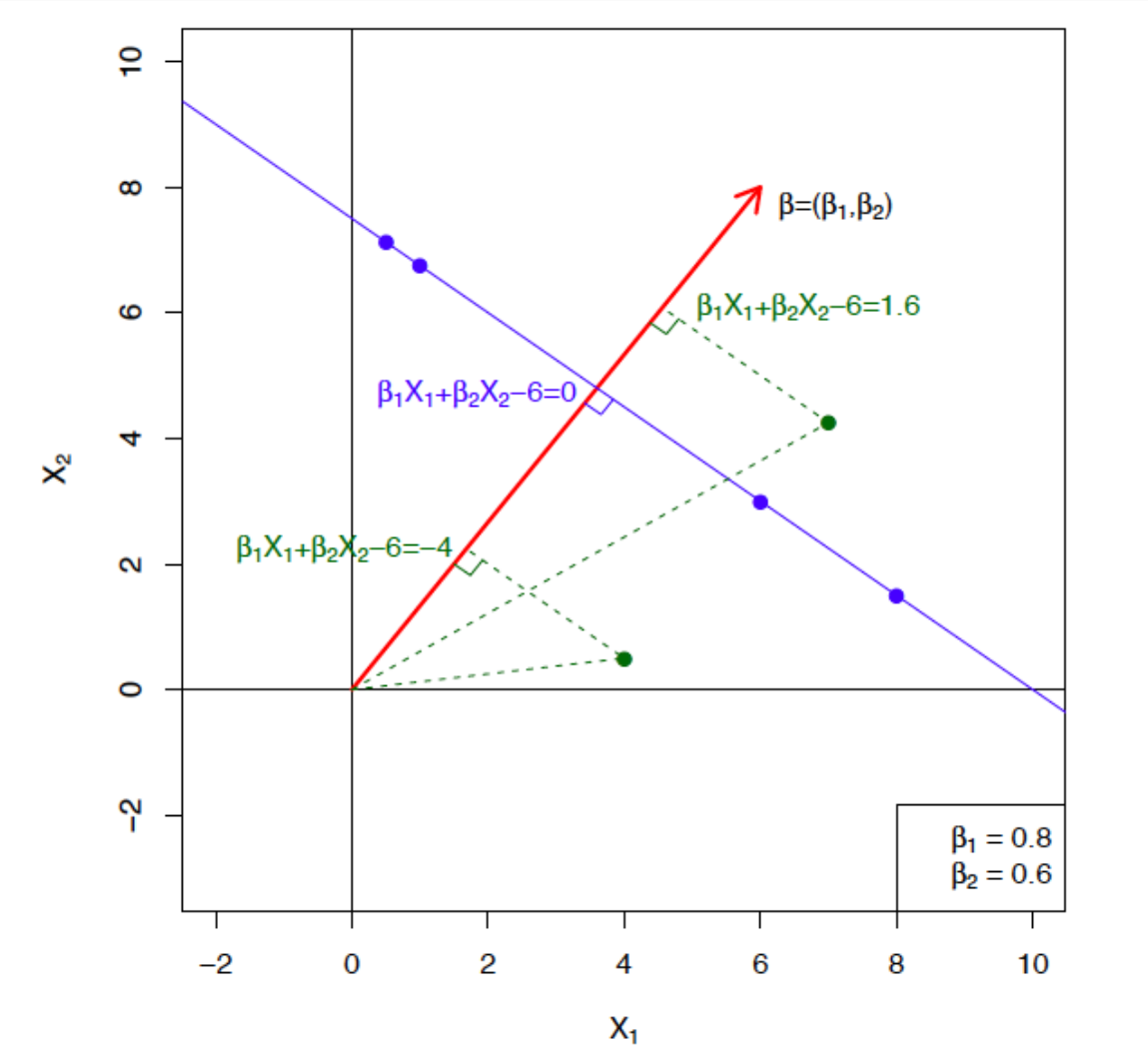
What is a hyperplane?

- A hyperplane in p dimensions is a flat affine subspace of dimension $p - 1$.
- In general the equation for a hyperplane has the form

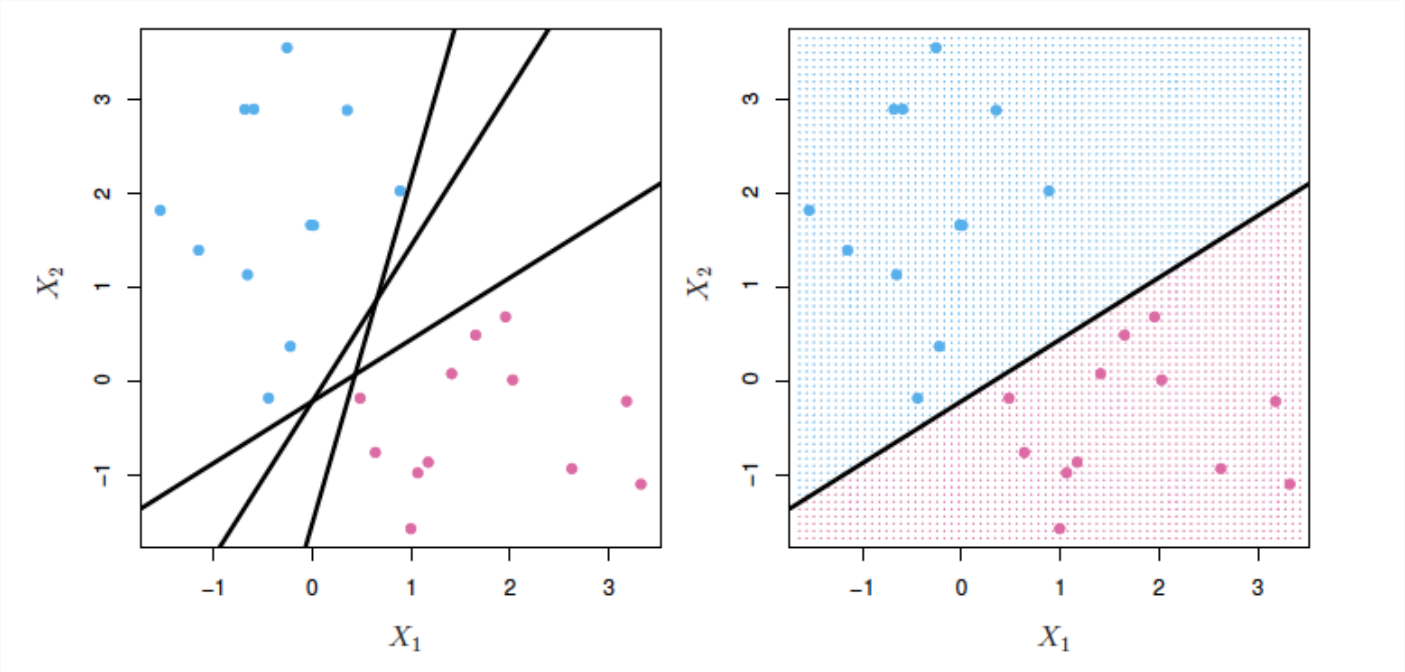
$$\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = 0$$

- In $p = 2$ dimensions a hyperplane is a line.
- If $\beta_0 = 0$, the hyperplane goes through the origin, otherwise not.
- The vector $\beta = (\beta_1, \ldots, \beta_p)$ is called the normal vector — it points in a direction orthogonal to the surface of a hyperplane.

Hyperplane in 2 dimensions



Separating hyperplanes



- If $f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$, then $f(X) > 0$ for points on one side of the hyperplane, and $f(X) < 0$ for points on the other side.
- If we code the colored points as $Y_i = +1$ for blue, say $Y_i = -1$ for mauve, then if $Y_i \cdot f(X_i) > 0$ for all i. $f(X) = 0$ defines a separating hyperplane.

Preliminaries

- Training data: $(\mathbf{x}_i, y_i)$; $i = 1, \ldots, n$ with $\mathbf{x}_i \in \mathbb{R}^p$ and $y_i \in \{-1, +1\}$.
- Define a separating hyperplane

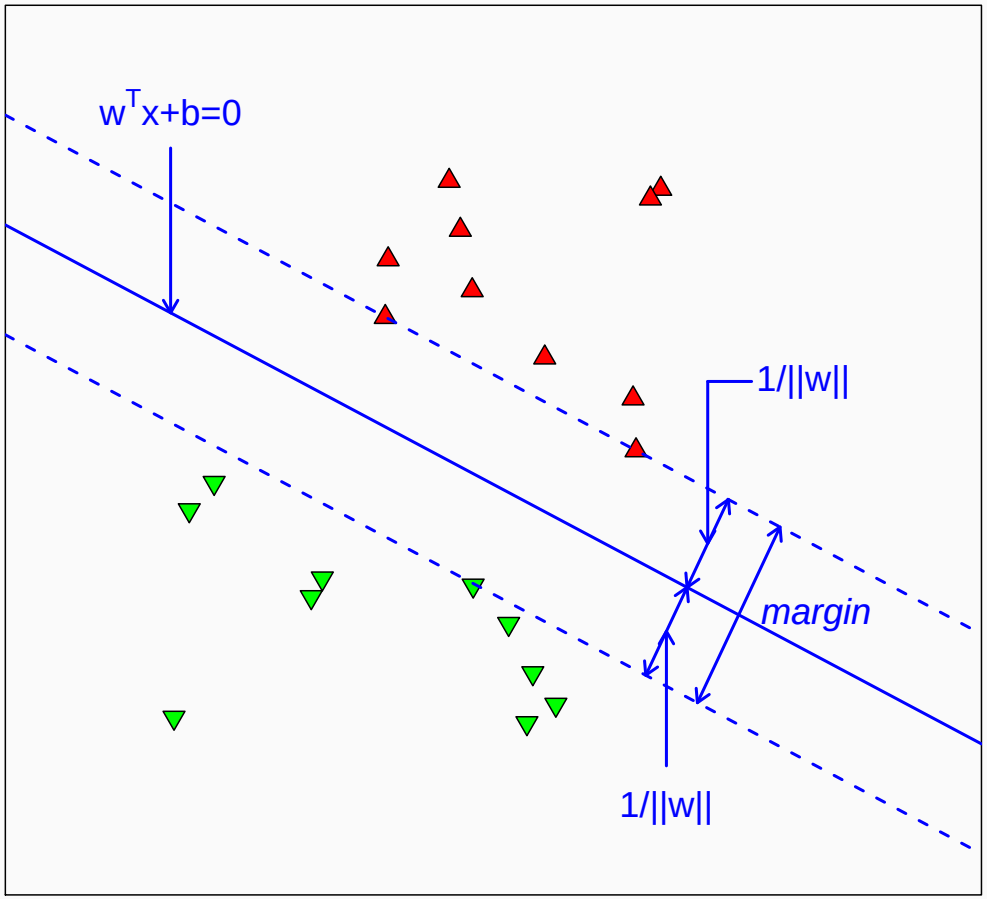
$$\{\mathbf{x} : f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = 0\}.$$

- A classification decision function is

$$G(\mathbf{x}) = \text{sign}(f(\mathbf{x})) = \text{sign}(\mathbf{w}^T x + b).$$

- A misclassified point $(\mathbf{x}, y) \Leftrightarrow yf(\mathbf{x}) < 0$, and vice versa.

Illustrative plot: separable



Recall that the distance between a point (x_0, y_0) to a line $Ax + By + C = 0$ is

$$\frac{Ax_0 + By_0 + C}{\sqrt{A^2 + B^2}}.$$

So the distance between $\mathbf{w}^T x + b = 1$ and $\mathbf{w}^T x + b = -1$ is $2/\| \mathbf{w} \|$.

In order to maximize the margin, we thus need to minimize $\| \mathbf{w} \|$.

The optimization problem can be rephrased as

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \| \mathbf{w} \| \text{ or } \frac{1}{2} \| \mathbf{w} \|^2 \\ \text{subject to} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1; \ i = 1, \ldots, n, \end{aligned}$$

where the two dashed hyperplanes are set as $\mathbf{w}^T \mathbf{x} + b = \pm 1$, denoted by H_1 and H_2 respectively.

In order to maximize the margin, we thus need to minimize $\| \mathbf{w} \|$, with the condition that there are no data points between H_1 and H_2.

This can be rephrased as a convex quadratic program, and solved efficiently.

This is a constrained optimization problem. It can be solved by the Lagrangian multiplier method.

Because it is quadratic, the surface is a paraboloid, with just a single global minimum (thus avoiding a problem we have with neural nets!)

So in the SVM problem the Langrangian is

$$\min L_p = \frac{1}{2}\| \mathbf{w} \|^2 - \sum_{i=1}^{n} a_i y_i (\mathbf{w}^T \mathbf{x}_i + b) + \sum_{i=1}^{n} a_i$$

s.t. $\forall i,\ a_i \geq 0$.

From the property that the derivatives at min $= 0$, we ge

$$\frac{\partial L_p}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n} a_i y_i \mathbf{x}_i = 0$$

$$\frac{\partial L_p}{\partial b} = \sum_{i=1}^{n} a_i y_i = 0$$

So

$$\mathbf{w} = \sum_{i=1}^{n} a_i y_i \mathbf{x}_i, \qquad \sum_{i=1}^{n} a_i y_i = 0$$

What's with this L_p business?

- This indicates that this is the primal form of the optimization problem
- We will actually solve the optimization problem by now solving for the dual of this original problem
- What is this dual formulation?

The Lagrangian Dual Problem: instead of minimizing over $\mathbf{w}, b$, subject to constraints involving a's, we can maximize over a (the dual variable) subject to the relations obtained previously for $\mathbf{w}$ and b

Our solution must satisfy these two relations:

$$\mathbf{w} = \sum_{i=1}^{n} a_i y_i \mathbf{x}_i, \qquad \sum_{i=1}^{n} a_i y_i = 0$$

By substituting for $\mathbf{w}$ and b back in the original eqn we can get rid of the dependence on $\mathbf{w}$ and b.

Note first that we already now have our answer for what the weights $\mathbf{w}$ must be: they are a linear combination of the training inputs and the training outputs, $\mathbf{x}_i$ and y_i, and the values of a. We will now solve for the a's by differentiating the dual problem wrt a, and setting it to zero. Most of the a's will turn out to have the value zero. The non-zero a's will correspond to the support vectors.

Dual problem:

$$\max L_D(a) = \sum_{i=1}^{n} a_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} a_i a_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$

s.t.

$$\sum_{i=1}^{n} a_i y_i = 0, \quad a_i \geq 0$$

(note that we have removed the dependence on $\mathbf{w}$ and b)

The L_D is a convex quadratic programming problem due to the $a_i a_j$ term and can be solved using standard optimization techniques.

The dual problem

- Kuhn-Tucker theorem: the solution we find here will be the same as the solution to the original problem
- Question: But why are we doing this???? (why not just solve the original problem????)
- Ans: Because this will let us solve the problem by computing the just the inner products of $\mathbf{x}_i, \mathbf{x}_j$ (which will be very important later on when we want to solve non-linearly separable classification problems)

Dual problem:

$$\max L_D(a_i) = \sum_{i=1}^{n} a_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} a_i a_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$

s.t.

$$\sum_{i=1}^{n} a_i y_i = 0, \quad a_i \geq 0$$

Notice that all we have are the dot products of $\mathbf{x}_i, \mathbf{x}_j$

Now knowing the a_i we can find the weights $\mathbf{w}$ for the maximal margin separating hyperplane

$$\mathbf{w} = \sum_{i=1}^{n} a_i y_i \mathbf{x}_i$$

And now, after training and finding the $\mathbf{w}$ by this method, given an unknown point u measured on features $\mathbf{x}_i$ we can classify it by looking at the sign of:

$$f(u) = \mathbf{w} \cdot u + b = \sum_{i=1}^{n} a_i y_i (\mathbf{x}_i \cdot u) + b$$

How to estimate b?

The derivatives have nothing to do with b! This is saying the value of b does not affect the minimum value of L_p. So we can choose an arbitrary value for b?

Not really. Because minimizing $L_p(w, b, a)$ is not the only goal b is involved in. It is also involved in the constraint,

$$y_i(\mathbf{w}_*^T x_i + b) \geq 1, \quad i = 1, \ldots, n$$

The closest positive and negative examples to the separating hyperplane are,

$$\arg \max_{i:y_i=-1} \mathbf{w}_*^T x_i, \qquad \arg \min_{i:y_i=+1} \mathbf{w}_*^T x_i$$

These verify (because they must be support vectors) the equations for the decision boundaries, that is,

$$\max_{i:y_i=-1} \mathbf{w}_*^T x_i + b = -1, \qquad \min_{i:y_i=+1} \mathbf{w}_*^T x_i + b = +1$$

Add the two and solve for b:

$$b_* = \frac{\max_{i:y_i=-1} \mathbf{w}_*^T x_i + \min_{i:y_i=+1} \mathbf{w}_*^T x_i}{2}$$

Inner products, similarity, and SVMs

Why should inner product kernels be involved in pattern recognition using SVMs, or at all?

— Intuition is that inner products provide some measure of 'similarity'

— Inner product in 2D between 2 vectors of unit length returns the cosine of the angle between them = how 'far apart' they are

Insight into inner products

$$\max L_D(a_i) = \sum_{i=1}^{n} a_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} a_i a_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$

s.t.

$$\sum_{i=1}^{n} a_i y_i = 0, \quad a_i \geq 0$$

The claim is that this function will be maximized if we give nonzero values to a's that correspond to the support vectors, i.e., those that 'matter' in fixing the maximum width margin ('street'). Well, consider what this looks like. Note first from the constraint condition that all the a's are positive. Now let's think about a few cases.

Case 1. If two features $\mathbf{x}_i, \mathbf{x}_j$ are completely dissimilar, their dot product is 0, and they don't contribute to L.

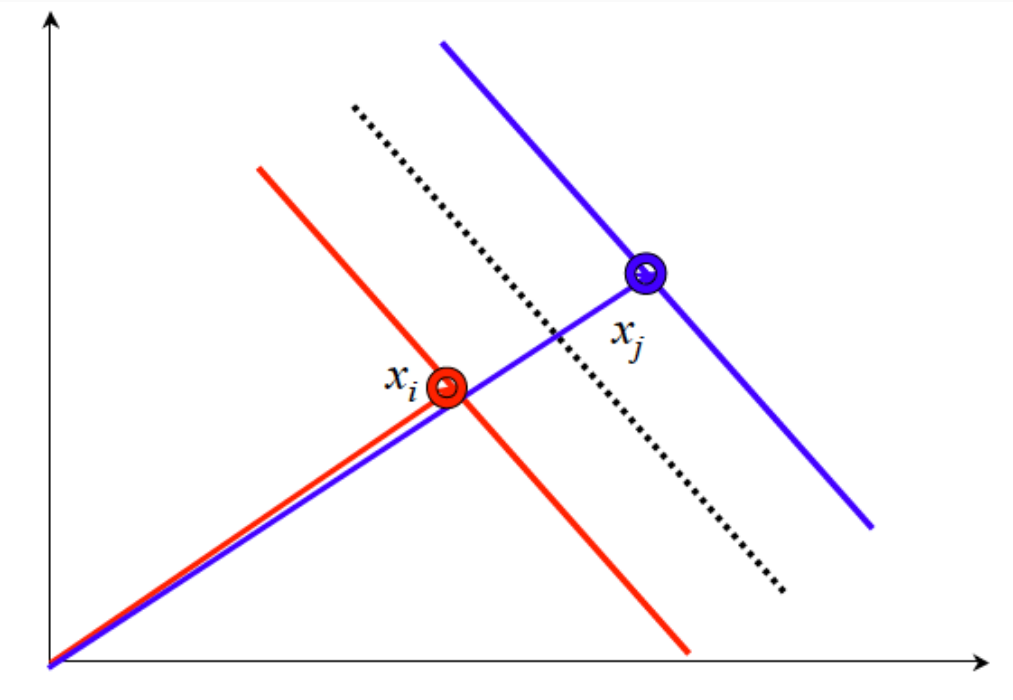
Case 2. If two features $\mathbf{x}_i, \mathbf{x}_j$ are completely alike, their dot product is 1. There are 2 subcases.

Subcase 1: both $\mathbf{x}_i$,and $\mathbf{x}_j$ predict the same output value y_i (either $+1$ or -1). Then $y_i y_j$ is always 1, and the value of $a_i a_j y_i y_j \mathbf{x}_i \mathbf{x}_j$ will be positive. But this would decrease the value of L (since it would subtract from the first term sum). So, the algorithm downgrades similar feature vectors that make the same prediction.

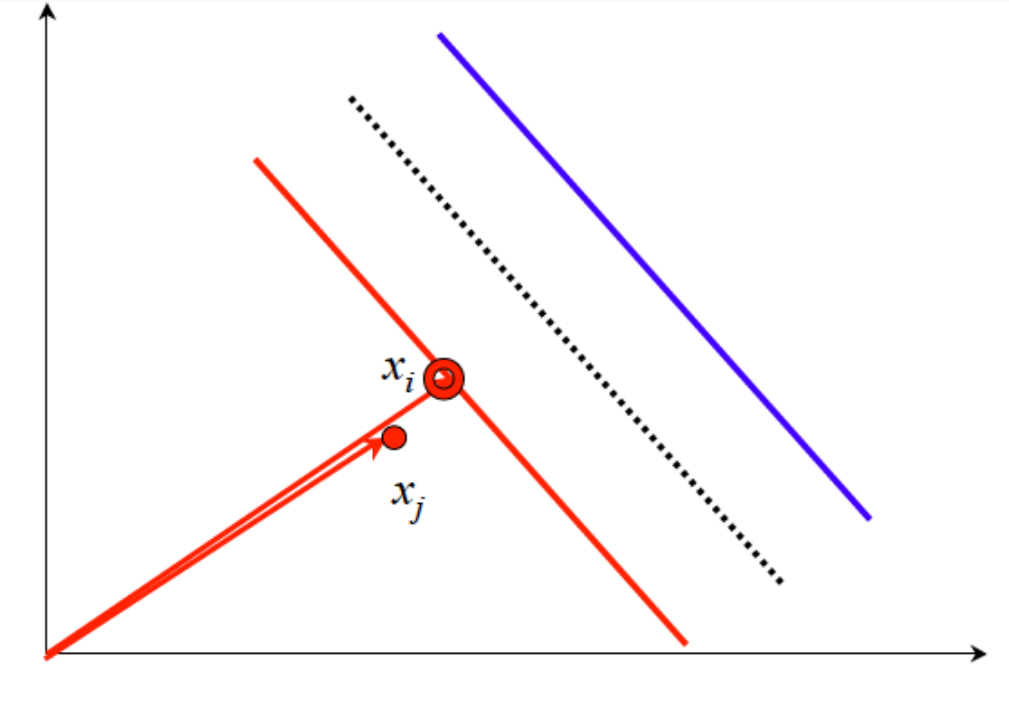
Subcase 2: $\mathbf{x}_i$ and $\mathbf{x}_j$ make opposite predictions about the output value y_i (i.e., one is $+1$, the other -1), but are otherwise very closely similar: then the product $a_i a_j y_i y_j \mathbf{x}_i \mathbf{x}_j$ is negative and we are subtracting it, so this adds to the sum, maximizing it. This is precisely the examples we are looking for: the critical ones that tell the two classes apart.

Insight into inner products, graphically

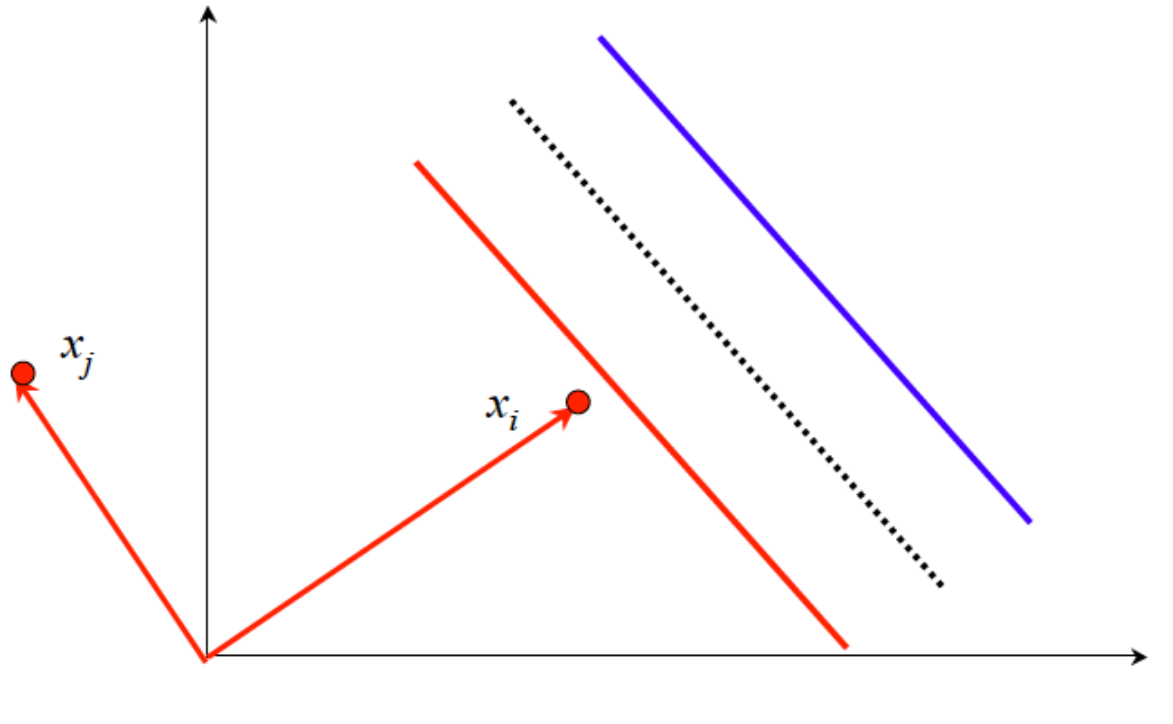
2 very very similar x_i, x_j vectors that predict difft classes tend to maximize the margin width



2 vectors that are similar but predict the same class are redundant

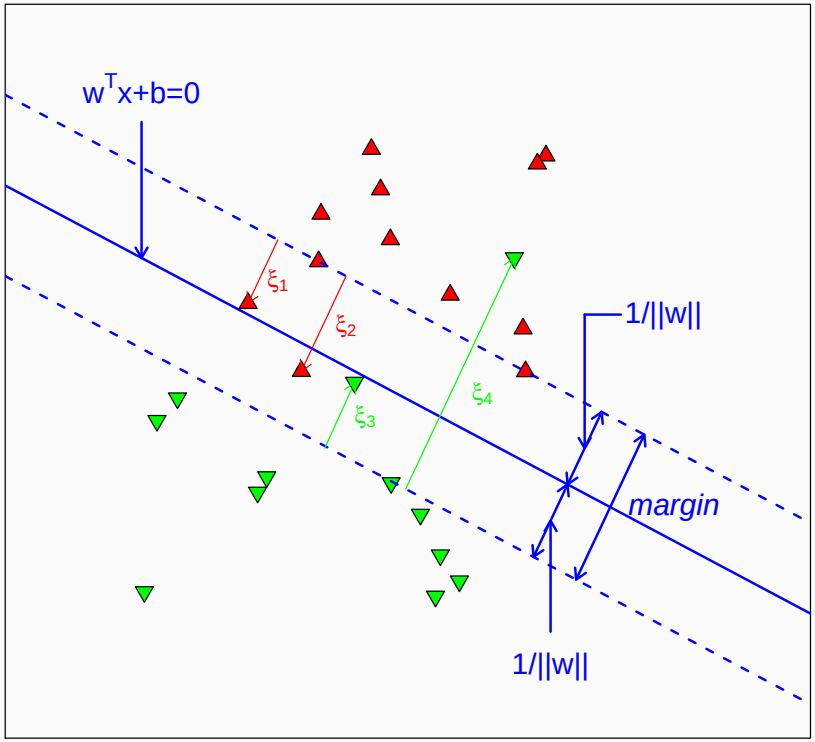


2 dissimilar (orthogonal) vectors don't count at all



Are we done???

Illustrative plot: nonseparable



Sometimes the data are separable, but noisy. This can lead to a poor solution for the maximal-margin classifier.

The support vector classifier maximizes a soft margin.

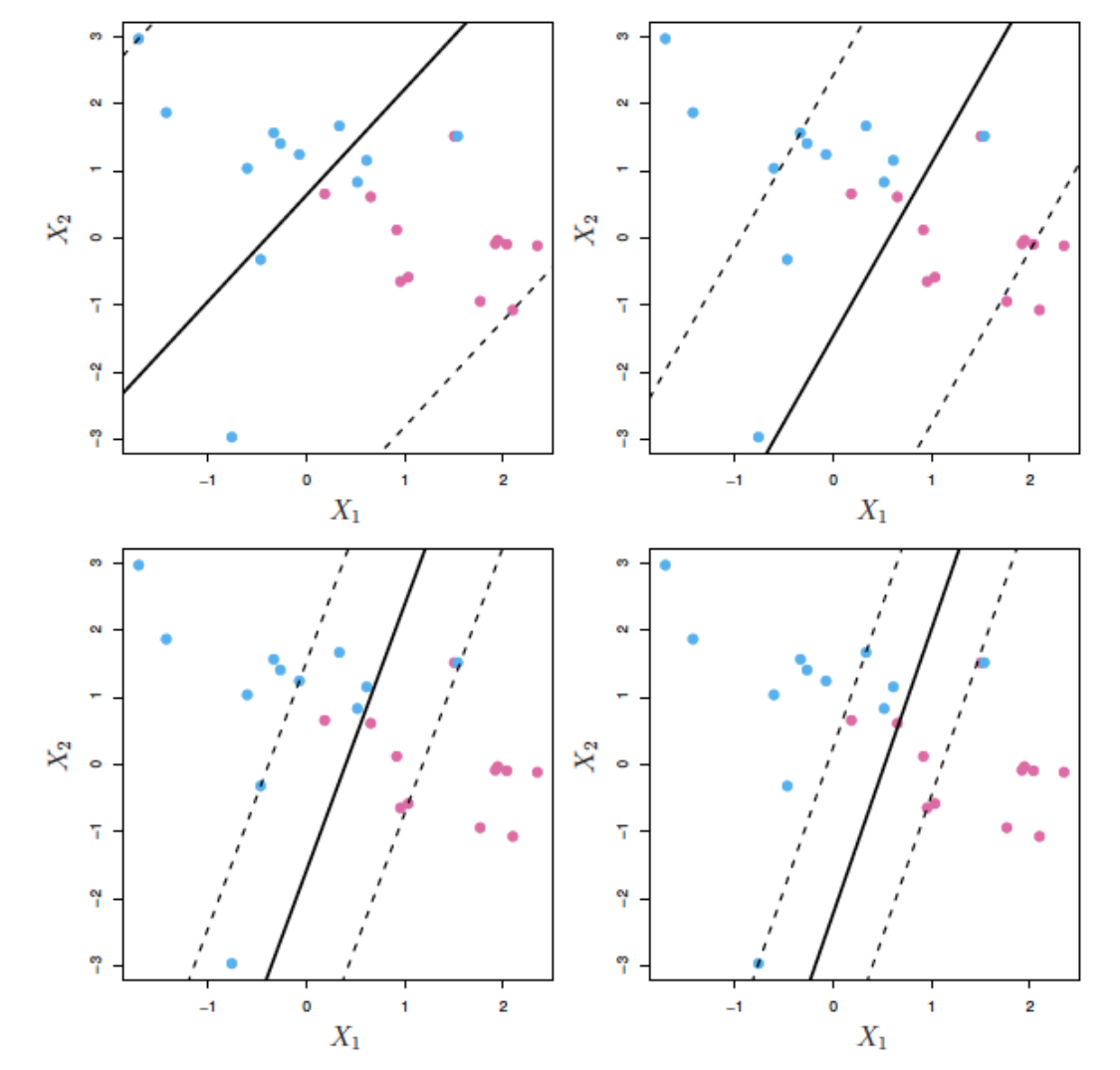
SVM formulation

When the data are not separable, the constraints need to be relaxed by introducing some slack variables ξ_i.

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\| \mathbf{w} \|^2 \\ \text{subject to} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i; \;\; i = 1, \ldots, n, \\ & \xi_i \geq 0, \; \sum_{i=1}^{n} \xi_i \leq s. \end{aligned}$$

This is the standard SVM formulation, where s controls the size of slack variables, or the severity of the misclassified observations.

s is a regularization parameter



Optimization

First the SVM formulation is equivalent to

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\| \mathbf{w} \|^2 + C \sum_{i=1}^{n} \xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i;\ \xi_i \geq 0;\ i = 1, \ldots, n. \end{aligned}$$

Lagrangian (primal) function is

$$\begin{aligned} L_P = & \frac{1}{2}\| \mathbf{w} \|^2 + C \sum_{i=1}^{n} \xi_i \\ & - \sum_{i=1}^{n} \alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i) - \sum_{i=1}^{n} \beta_i \xi_i, \end{aligned} \tag{1}$$

which we minimize w.r.t. $\mathbf{w}$, b and ξ_i, subject to $\xi_i, \alpha_i, \beta_i \geq 0$.

Dual form

Setting the derivatives to zero,

$$w = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i, \; 0 = \sum_{i=1}^{n} \alpha_i y_i, \; C = \alpha_i + \beta_i,$$

as well as $\xi_i, \alpha_i, \beta_i \geq 0$.

Substituting them back to the primal function yields the dual function,

$$\max_{\alpha} \quad L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{i'=1}^{n} \alpha_i \alpha_{i'} y_i y_{i'} \langle \mathbf{x}_i, \mathbf{x}_{i'} \rangle$$

$$\text{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0; \; 0 \leq \alpha_i \leq C.$$

Remarks

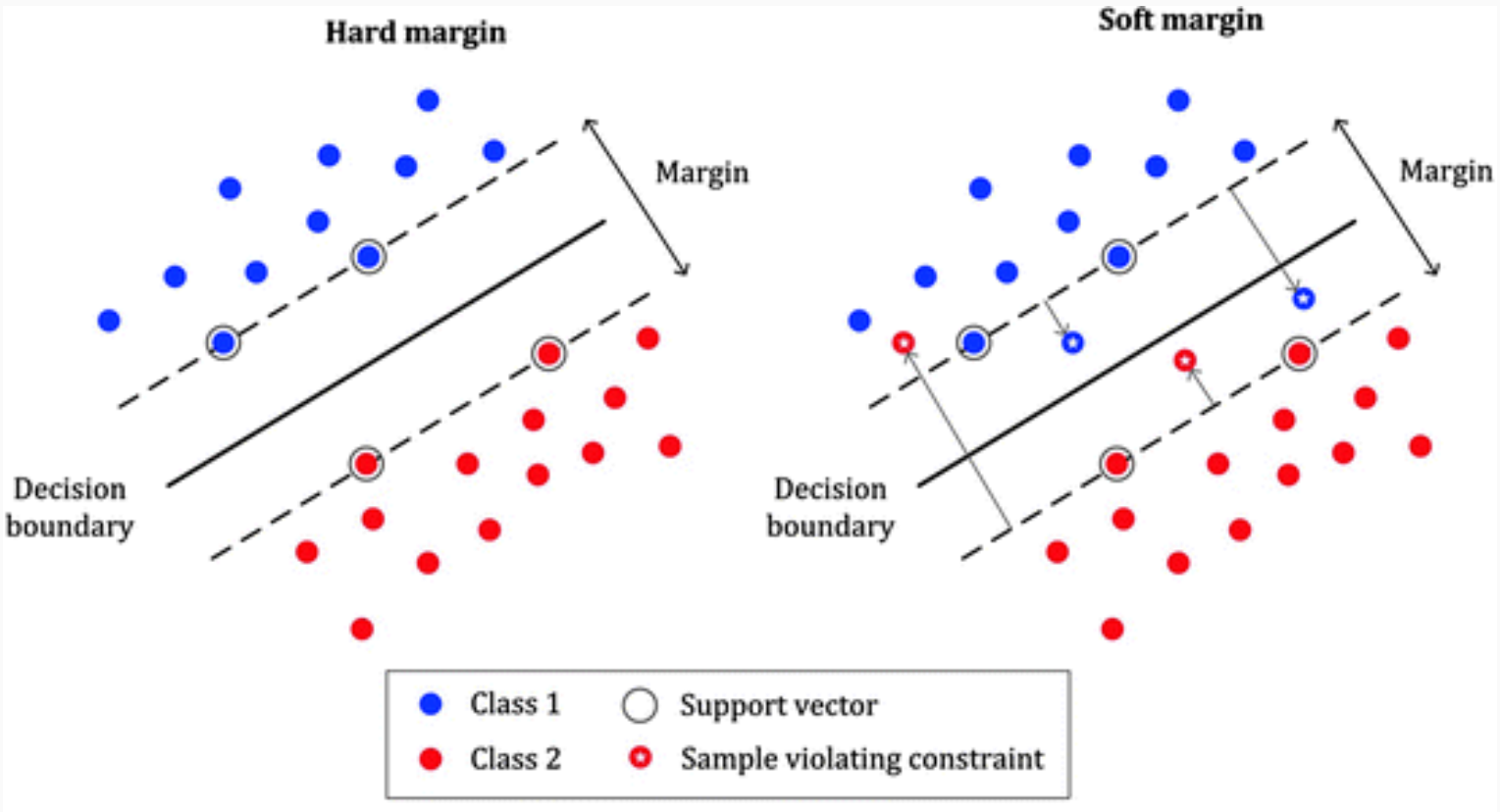
- The solutions to the primal and the dual forms are equivalent,

$$\hat{w} = \sum_{i=1}^{n} \hat{\alpha}_i y_i \mathbf{x}_i.$$

- The solutions to the primal and the dual forms must satisfy the Karush-Kuhn-Tucker conditions:

$$\begin{aligned} \alpha_i(y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i) &= 0, \\ \beta_i\xi_i &= 0, \\ y_i(\mathbf{w}^T\mathbf{x}_i + b) &\geq 1 - \xi_i. \end{aligned}$$

$$\alpha_i \geq 0, \quad \beta_i \geq 0$$


Hard margin
Margin
Decision boundary
Soft margin
Margin
Decision boundary
Class 1
Class 2
Support vector
Sample violating constraint

More on KKT conditions

- $\hat{\alpha}_i > 0$ only when $y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1 - \xi_i$, or equivalently,

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) < 1.$$

  These points are called *support vectors*.

- Among support vectors, some have $\hat{\xi}_i > 0$ and $\hat{\alpha}_i = C$ (Non-margin SVs); and others lie on the margin $\hat{\xi}_i = 0$, and thus $0 < \hat{\alpha}_i \leq C$ (Margin SV).

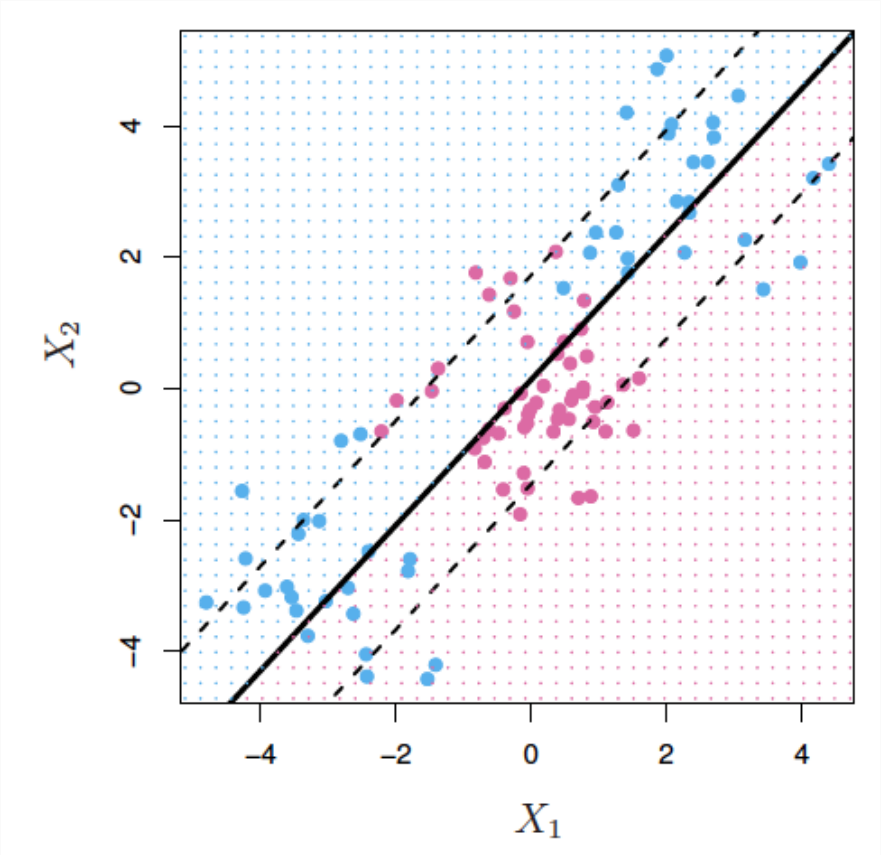
- The intercept b cannot be obtained from the dual form, but can be solved by using any point with $0 < \hat{\alpha}_i < C$, where

$$y_i(\hat{\mathbf{w}}^T\mathbf{x}_i + b) = 1.$$

  All points with $0 < \hat{\alpha}_i < C$ should yield the same $\hat{b}$. If no point with $0 < \hat{\alpha}_i < C$, plugging $\hat{\mathbf{w}}_i$ back into the primal form and solve for $\hat{b}$.

Nonlinear SVM

Linear boundary can fail

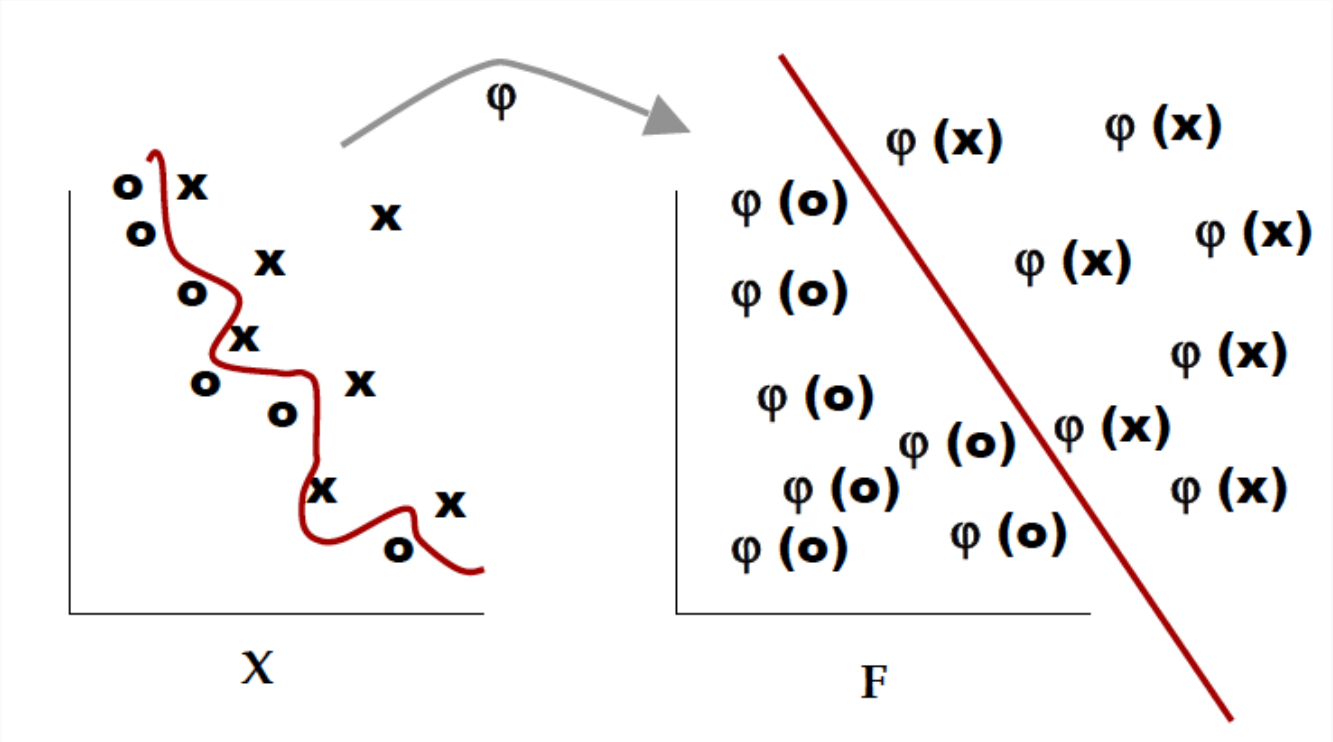


Sometime a linear boundary simply won't work, no matter what value of C or s.

The example on the above is such a case.

What to do?

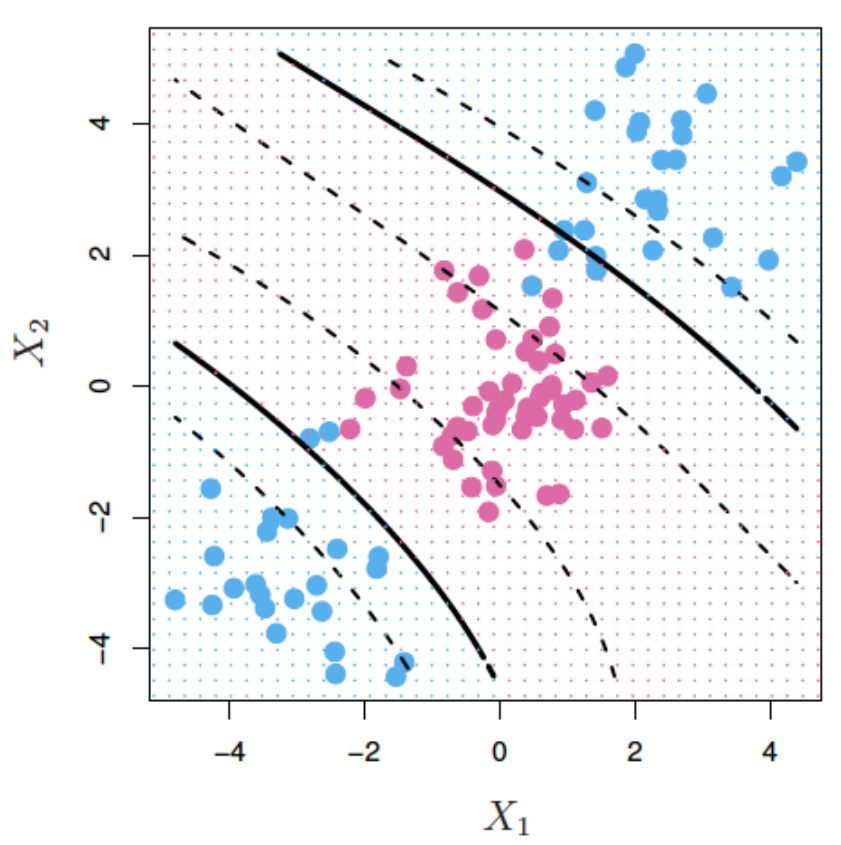
Linear boundary can fail



Feature expansion

- Enlarge the space of features by including transformations; e.g. $X_1^2, X_1^3, X_1X_2, \ldots$ Hence go from a p-dimensional space to a $M > p$ dimensional space.
- Fit a support-vector classifier in the enlarged space.
- This results in non-linear decision boundaries in the original space.

Cubic polynomials



Here we use a basis expansion of cubic polynomials; From 2 variables to 9

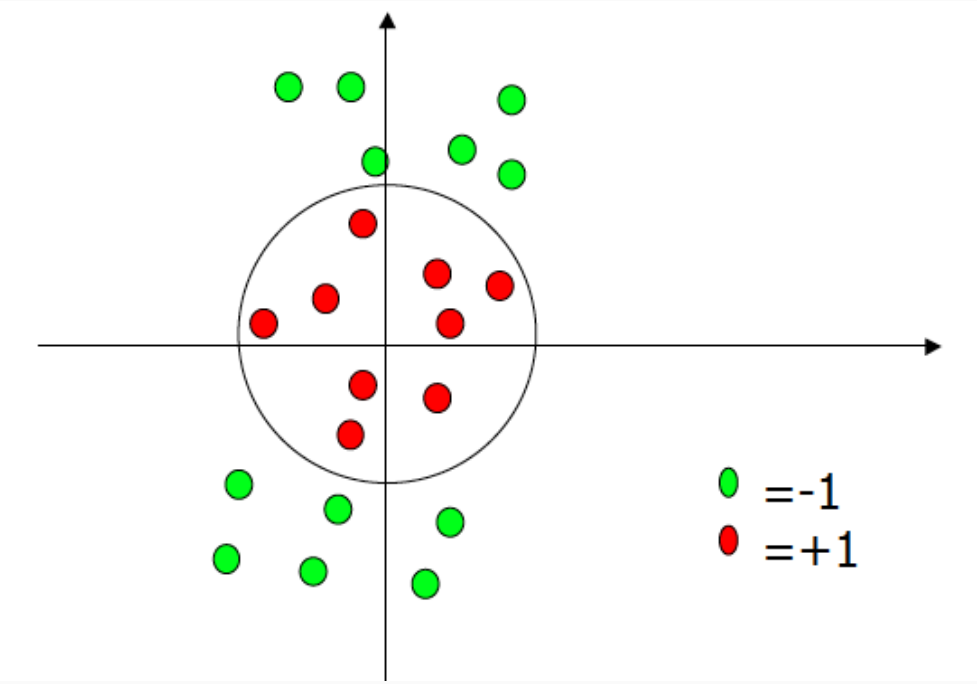
The support-vector classifier in the enlarged space solves the problem in the lower-dimensional space

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \beta_6 X_1^3 + \beta_7 X_2^3 + \beta_8 X_1 X_2^2 + \beta_9 X_1^2 X_2 = 0$$

Nonlinearities and kernels

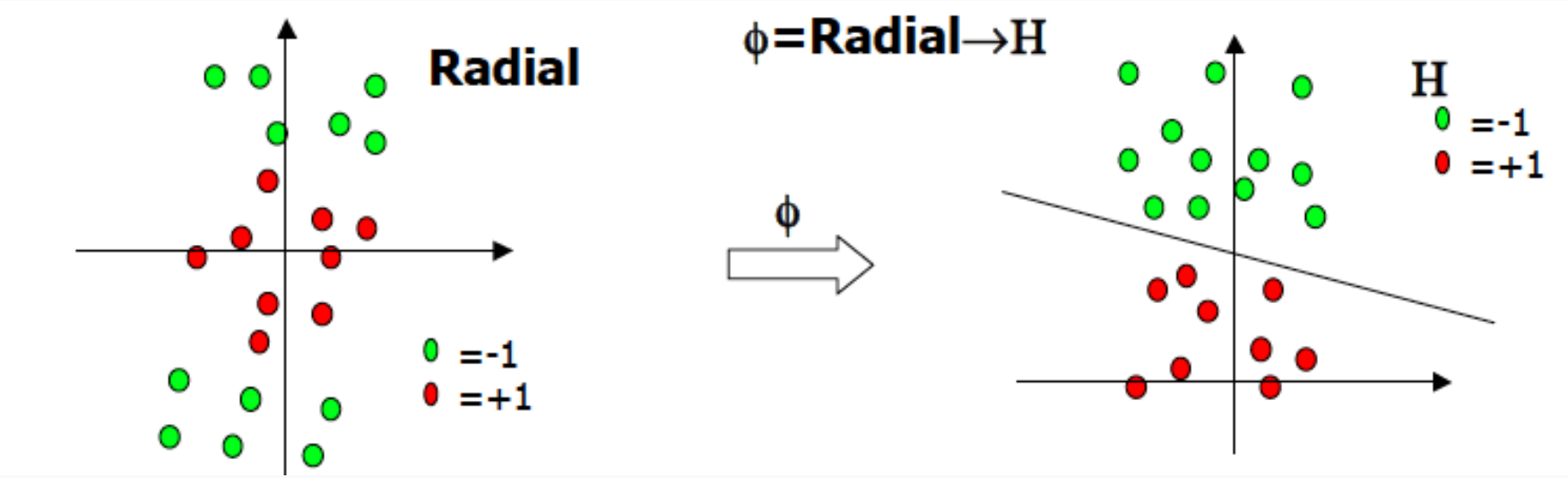
- Polynomials (especially high-dimensional ones) get wild rather fast.
- There is a more elegant and controlled way to introduce nonlinearities in support-vector classifiers — through the use of kernels.
- Before we discuss these, we must understand the role of inner products in support-vector classifiers.

Problems with linear SVM



What transform would separate these?

Imagine a function ϕ that maps the data into another space:



Nonlinear SVM with kernels

The key idea of extending linear SVM, and many other linear procedures, to nonlinear is to:

- Enlarge the predictor space using basis expansion functions $h_1(\mathbf{x}), \ldots, h_M(\mathbf{x})$
- Construct a linear separating hyperplane $f(x) = \mathbf{w}^T h(\mathbf{x}) + b$ in the enlarged space for better training performance
- The linear separating hyperplane in the enlarged space can be translated into a nonlinear separating hyperplane in the original space.

In the enlarged space, the dual form becomes

$$\max_{\alpha} \quad L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{i'=1}^{n} \alpha_i \alpha_{i'} y_i y_{i'} \langle h(\mathbf{x}_i), h(\mathbf{x}_{i'}) \rangle$$

$$\text{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0; \ 0 \leq \alpha_i \leq C.$$

The solution is $\hat{\mathbf{w}} = \sum_{i=1}^{n} \hat{\alpha}_i y_i h(\mathbf{x}_i)$, and

$$\hat{f}(\mathbf{x}) = \hat{\mathbf{w}}^T h(\mathbf{x}) + \hat{b} = \sum_{i=1}^{n} \hat{\alpha}_i y_i \langle h(\mathbf{x}_i), h(\mathbf{x}) \rangle + \hat{b}.$$

The interesting part is the formulation relies on $h(\mathbf{x})$ only through their inner products.

Kernels

Define

$$K(\mathbf{x}, \mathbf{x}') = \langle h(\mathbf{x}), h(\mathbf{x}') \rangle,$$

and thus we need not specify $h(\cdot)$ at all, but only $K(\cdot, \cdot)$.

Popular choice of $K(\cdot, \cdot)$:

- Linear: $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$
- Degree-d polynomial: $K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T \mathbf{x}')^d$
- Radial (Gaussian): $K(\mathbf{x}, \mathbf{x}') = \exp(-\| \mathbf{x} - \mathbf{x}' \|^2 / \sigma^2)$

SVM in a regularization form

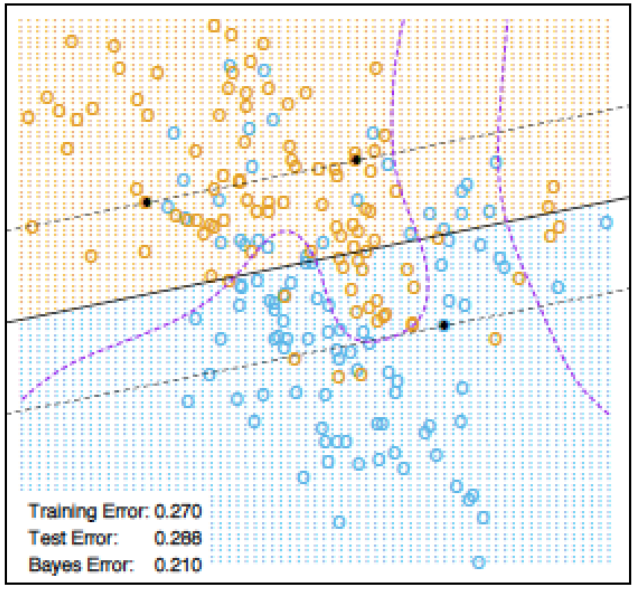
A generic regularization form

$$\min_{f \in \mathcal{F}} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i)) + \lambda J(f).$$

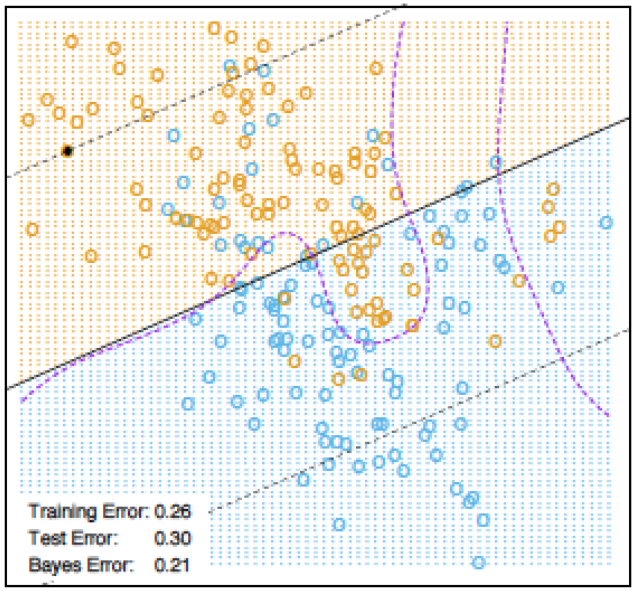
For SVM, note that the solution of ξ_i must be $\xi_i = (1 - y_i f(\mathbf{x}_i))_+$ with $z_+ = \min(z, 0)$. Therefore, the SVM formulation can be rephrased as

$$\min_{f \in \mathcal{F}} \sum_{i=1}^{n} (1 - y_i f(\mathbf{x}_i))_+ + \frac{\lambda}{2} \|f\|_{\mathcal{F}}^2,$$

where $\mathcal{F}$ is the candidate functional space, $L(z) = (1 - z)_+$ is the *hinge loss*, and $\|\cdot\|_{\mathcal{F}}$ is the norm associated with $\mathcal{F}$.

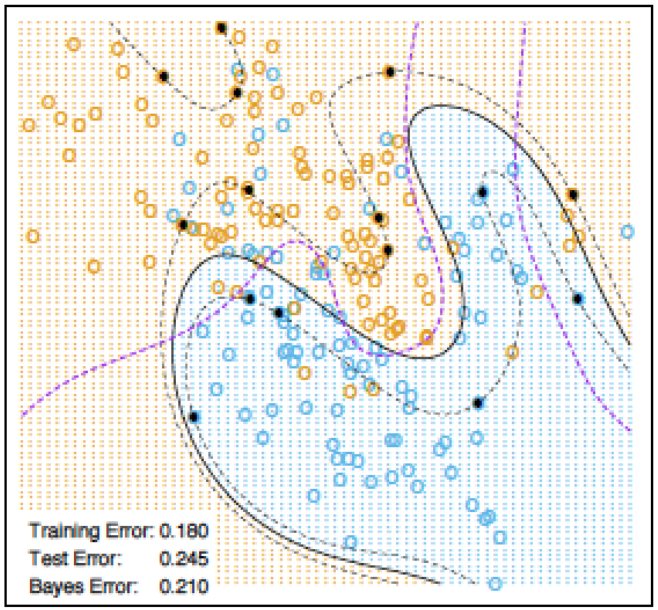


$C = 10000$

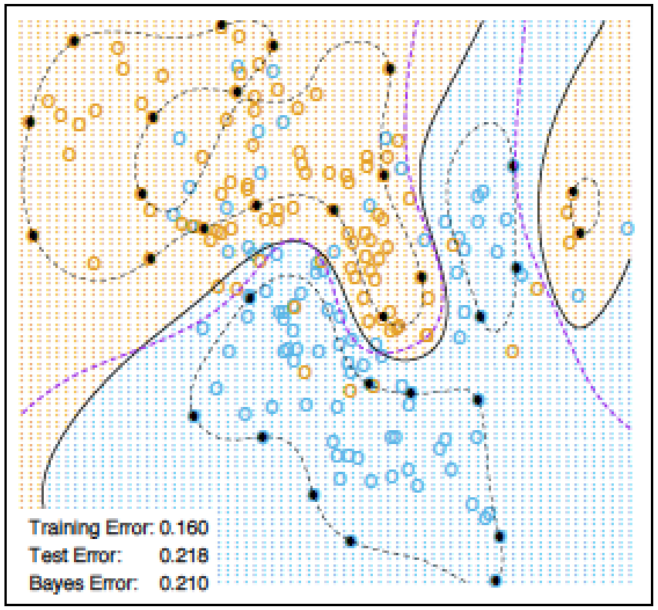


$C = 0.01$

SVM - Degree-4 Polynomial in Feature Space



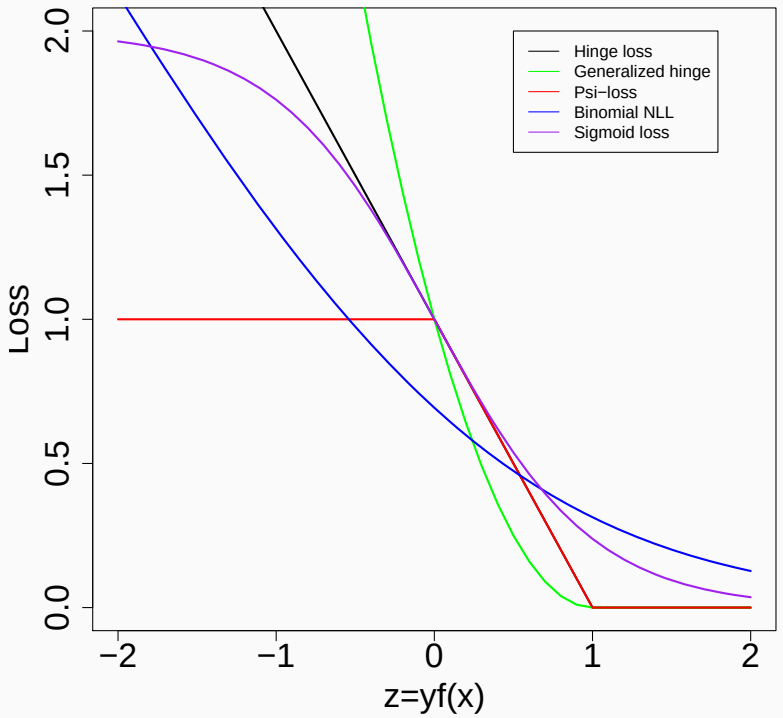
SVM - Radial Kernel in Feature Space



Left panel: The linear support vector boundary for the mixture data example with two overlapping classes, for two different values of C. The broken lines indicate the margins, where $f(x) = \pm 1$. The support points ($a_i > 0$) are all the points on the wrong side of their margin. The black solid dots are those support points falling exactly on the margin ($\xi_i = 0, a_i > 0$). In the upper panel 62% of the observations are support points, while in the lower panel 85% are. The broken purple curve in the background is the Bayes decision boundary

Right panel: Two nonlinear SVMs for the mixture data. The upper plot uses a 4th degree polynomial kernel, the lower a radial basis kernel. In each case C was tuned to approximately achieve the best test error performance, and $C = 1$ worked well in both cases. The radial basis kernel performs the best (close to Bayes optimal), as might be expected given the data arise from mixtures of Gaussians. The broken purple curve in the background is the Bayes decision boundary

Large margin loss $L(z)$



- Hinge loss (Vapnik, 1995):
  $L(z) = (1 - z)_+ = \max(1 - z, 0)$
- Generalized hinge loss (Lin, 2002):
  $L(z) = (1 - z)_+^q$
- ψ-loss (Shen, Tseng, Zhang and Wong, 2003):
  $L(z) = \psi(z) = \min(1, (1 - z)_+)$
- Binomial NLL (Zhu and Hastie, 2004):
  $L(z) = \log(1 + e^{-z})$
- Sigmoid loss (Mason, Bartlett and Baxter, 2000):
  $L(z) = 1 - \tanh(\lambda z)$

Fisher consistency

The hinge loss is Fisher consistent as the global minimizer of $\mathbb{E}(1 - Yf(\mathbf{X}))_+$ is exactly the Bayes rule

$$G^*(\mathbf{x}) = \text{sign}(Pr(Y = 1|\,\mathbf{X} = \mathbf{x}) - 1/2).$$

Compared with other losses as in Table 12.1,

- Other losses aim to estimate $Pr(Y = 1|\,\mathbf{X} = \mathbf{x})$, which contains more information than classification;
- Hinge loss aims to estimate $\text{sign}(Pr(Y = 1|\,\mathbf{X} = \mathbf{x}) - 1/2)$, which targets on classification directly.

SVM in RKHS

If $\mathcal{F}$ is set to be a RKHS $\mathcal{H}_K$ with a positive definite kernel $K(\cdot, \cdot)$, $f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i K(\mathbf{x}_i, \mathbf{x}) + b$, then

$$\|f\|_{\mathcal{H}_K}^2 = \alpha^T \mathbf{K} \alpha,$$

where $\mathbf{K} = (K(x_i, x_{i'})_{i,i'=1}^n$.

The kernel based SVM formulation then becomes

$$\min_{\alpha, b} \sum_{i=1}^{n} \Big(1 - y_i \Big(\sum_{j=1}^{n} \alpha_j K(\mathbf{x}_i, \mathbf{x}_j) + b \Big) \Big)_+ + \frac{\lambda}{2} \alpha^T \mathbf{K} \alpha.$$

Ψ-learning

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^{n} \Psi(y_i f(\mathbf{x}_i)) + \frac{\lambda}{2} \|f\|_K^2,$$

where the loss function

$$L(y, f) = \Psi(yf) = \max((1 - yf)_+, 1),$$

is also Fisher consistent.

- Pros: Ψ-loss is closer to 0-1 loss, and is robust to outliers
- Cons: Non-convex objective function
  - Non-convex optimization technique, such as difference convex algorithm (DCA), can be used

Kernel logistic regression

Replacing the hinge loss by the negative log-likelihood yields that

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^{n} (y_i f(\mathbf{x}_i) - \log(1 + \exp(f(\mathbf{x}_i)))) + \frac{\lambda}{2} \|f\|_K^2,$$

Newton-Raphson method updates α iteratively,

$$\alpha^{new} = (\mathbf{K}^T \mathbf{W} \mathbf{K} + \lambda \mathbf{K})^{-1} \mathbf{K}^T \mathbf{W} \mathbf{y}^{new},$$

where $\mathbf{W} = \text{diag}(\hat{p}_i(1 - \hat{p}_i))$, $\mathbf{y}^{new} = \mathbf{K}\alpha^{old} + \mathbf{W}^{-1}(\mathbf{y} - \hat{\mathbf{p}})$ with $\hat{\mathbf{p}} = (\hat{p}_1, \ldots, \hat{p}_n)$ and

$$\hat{p}_i = \widehat{Pr}(Y = 1 | \mathbf{X} = \mathbf{x}_i) = (1 + \exp(\hat{f}(\mathbf{x}_i)))^{-1}.$$

Estimating $Pr(Y = 1|x)$ for SVM

Question: Is it possible to still obtain $\widehat{Pr}(Y = 1|\,\mathbf{x})$ while attaining the high classification accuracy of SVM?

Solutions:

- Link function (Platt, 1998): $p(\mathbf{x}) = \frac{1}{1+\exp(Af(\mathbf{x})+B)}$ with $p(\mathbf{x}) = P(Y = 1|\,\mathbf{X} = \mathbf{x})$.
  - Lack of support for using such a link function.
- Sequential bracketing (Wang et al., 2006; Bka): using a sequence of weighted classification to locate the subinterval containing $p(\mathbf{x})$.
  - No assumption needed for the link function.

Weighted SVM

Different weights $(1-\pi, \pi)$ are associated with positive and negative cases,

$$\min_{f \in \mathcal{H}_K} \frac{1}{n} \left((1-\pi) \sum_{y_i=1} L(f(\mathbf{x}_i)) + \pi \sum_{y_i=-1} L(-f(\mathbf{x}_i)) \right) + \frac{\lambda}{2} \|f\|^2_{\mathcal{H}_K},$$

where $L(z) = (1-z)_+$ and can be other loss functions as well.

Lemma
Assume $L(z)$ is hinge loss, and $s(Y) = 1 - \pi$ if $Y = 1$, and π o.w.,

$$\underset{f}{\operatorname{argmin}}\ \mathbb{E}\Big(s(Y)L(Yf(\mathbf{X}))\Big) = \operatorname{sign}(f_\pi(\mathbf{x}))$$

where $f_\pi(\mathbf{x}) = Pr(Y = 1|\,\mathbf{x}) - \pi$.

- The weighted SVM approximates $\operatorname{sign}(f_\pi(\mathbf{x}))$
- $\operatorname{sign}(f_\pi(\mathbf{x})) = 1 \Leftrightarrow Pr(Y = 1|\,\mathbf{x}) > \pi$
- $f_\pi(\mathbf{x})$ and $\operatorname{sign}(f_\pi(\mathbf{x}))$ are nonincreasing in π

π	0	0.1	$\cdots$	0.6	0.7	$\cdots$	1
$\operatorname{sign}(f_\pi(\mathbf{x}))$	+1	+1	$\cdots$	+1	-1	$\cdots$	-1

Algorithm

Step 1. Initialize $\pi_j = (j-1)/m$; $j = 1, \cdots, m+1$.

Step 2. Train weighted margin classifier for π_j; $j = 1, \cdots, m+1$.

Step 3. Estimate labels of $\mathbf{x}$ by $\mathrm{sign}(\hat{f}_{\pi_j}(\mathbf{x}))$.

Step 4. Sort $\mathrm{sign}(\hat{f}_{\pi_j}(\mathbf{x}))$; $j = 1, \cdots, m+1$, to compute

$$\begin{aligned} \pi^* &= \max\{\pi_j : \mathrm{sign}(\hat{f}_{\pi_j}(\mathbf{x})) = +1\}, \\ \pi_* &= \min\{\pi_j : \mathrm{sign}(\hat{f}_{\pi_j}(\mathbf{x})) = -1\}. \end{aligned}$$

Then (π^*, π_*) or (π_*, π^*) is the interval containing $p(\mathbf{x})$, and thus the estimated class probability is

$$\widehat{p}(\mathbf{x}) = \frac{1}{2}(\pi^* + \pi_*).$$

Multiclass SVM

More than 2 classes?

The SVM as defined works for $K = 2$ classes. What do we do if we have $K > 2$ classes?

OVA: One versus All. Fit K different 2-class SVM classifiers f_k, $k = 1, \ldots, K$; each class versus the rest. Classify x^* to the class for which $f_k(x^*)$ is the largest.

OVO: One versus One. Fit all $\binom{K}{2}$ pairwise classifiers $f_{k\ell}$. Classify x^* to the class that wins the most pairwise competitions.

Which to choose? If K is not too large, use OVO.

Multiclass SVM

Training sample $(\mathbf{x}_i, y_i)_{i=1}^n$ with $\mathbf{x}_i \in \mathbb{R}^p$ and $y_i \in \{1, \ldots, K\}$. Denote $\mathbf{f} = (f_1, \ldots, f_K)^T$ with $\sum_k f_k(\mathbf{x}) = 0$.

$$\min_{f \in \mathcal{F}} \sum_{i=1}^{n} L(y_i, \mathbf{f}(\mathbf{x}_i)) + \lambda J(\mathbf{f})$$

- One-vs-rest

$$\hat{y} = \arg\max_k w_k^T x \quad \text{(prediction rule)}$$

$$w_y^T x \geq w_k^T x + 1 - \xi \quad \forall\ k \neq y$$

$$L(y, \mathbf{f}(\mathbf{x})) = \sum_{k \neq y} (1 - (f_y(\mathbf{x}) - f_k(\mathbf{x})))_+$$

Support vector regression

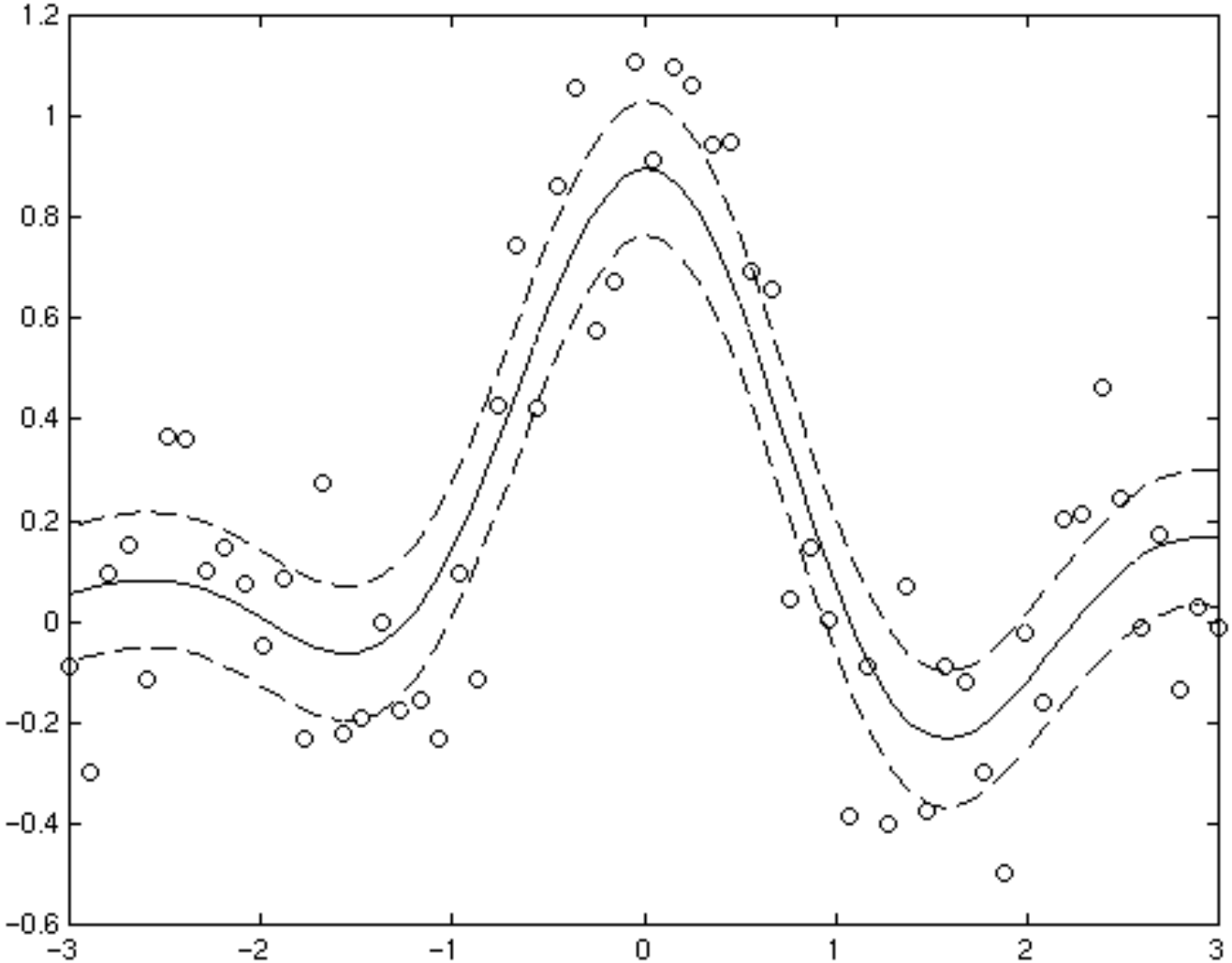
SVR is formulated as:

$$\begin{aligned} \min_{f \in \mathcal{H}_K} \quad & \sum_{i=1}^{n} \xi_i + \frac{\lambda}{2} \|f\|^2_{\mathcal{H}_K} \\ \text{subject to} \quad & |y_i - f(\mathbf{x}_i))| \leq \epsilon + \xi_i; \ \xi_i \geq 0; \ i = 1, \ldots, n. \end{aligned}$$

Some remarks:

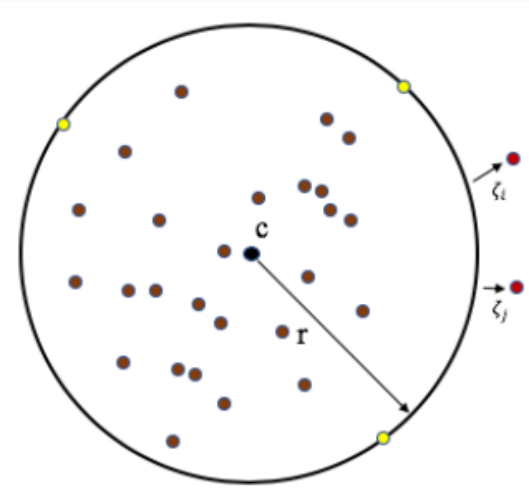
- Similar as robust regression
- Require additional effort for the optimization
- ϵ is an additional tolerance parameter that needs to be tuned

Support vector regression



One-class SVM

Can we learn from examples of just one class, say positive examples?



One-class SVM

The algorithm obtains a spherical boundary, in feature space, around the data. The volume of this hypersphere is minimized, to minimize the effect of incorporating outliers in the solution.

The resulting hypersphere is characterized by a center μ and a radius $R > 0$ as distance from the center to (any support vector on) the boundary, of which the volume R^2 will be minimized. The center μ is a linear combination of the support vectors (that are the training data points for which the Lagrange multiplier is non-zero). Just as the traditional formulation, it could be required that all the distances from data points x_i to the center is strict less then R, but to create a soft margin again slack variables ξ with penalty parameter C are used.

$$\begin{aligned} \min_{\mu, R, \xi} \quad & R^2 + C \sum_{i=1}^{n} \xi_i \\ \text{subject to} \quad & \|\mathbf{x}_i - \mu\|^2 \leq R^2 + \xi_i;\ \xi_i \geq 0;\ i = 1, \ldots, n. \end{aligned}$$

After solving this by introduction Lagrange multipliers a_i, a new data point z can be tested to be in or out of class.

Applications:

- Change point/outlier detection
- Cluster analysis

Summary

- A hugely (perhaps the most!) popular classification algorithm
- Reasonably mature, highly optimized SVM softwares freely available (perhaps the reason why it is more popular than various other competing algorithms) Some popular ones: libSVM, LIBLINEAR, scikit-learn also provides SVM
- Lots of work on scaling up SVMs (both large N and large p)
- Extensions beyond binary classification (e.g., multiclass, one-class, structured outputs)
- Can even be used for regression problems (Support Vector Regression)
- Nonlinear extensions possible via kernels